Thermal Radiance Texture for a Steel Bottle

A beginner's offline study guide: concepts, setup, step-by-step practice, testing and troubleshooting

Internship project: Spectral Radiance Texture Generation for LWIR Simulation in Unity
Platform: Windows (PowerShell and VS Code terminal)
Practice object: a steel bottle

> **TIP: How to use this guide.** Read Parts 1–3 once to understand the idea. Then follow Part 4 step by step. Part 4 is built in **two parallel tracks**: while a download or installation is running (Track A, blue), you prepare the steel bottle input data (Track B, green). You never sit and wait.

Contents

Part	Title	What you get
1	The big picture and basic concepts	What the task is, why, key words in plain English
2	The physics in simple words	Planck's law, emissivity, reflection, a worked example
3	The existing procedure (from your task document)	Pipeline, 6-month plan, success criteria
4	Hands-on: install and prepare data in parallel	Stages 1–5, exact commands, where to run them
5	The Python script and running it	Full working code, expected results
6	Testing, accuracy and performance	How to check results and speed up
7	Troubleshooting	Common errors and fixes
8	Suggested improvements (NOT in the task document)	Ideas and the best one for your guide
9	Summary and questions for your guide	One-page recap and 10 questions
A	Quick command reference	All commands with where to run them

Part 1. The big picture and basic concepts

1.1 What is the task?

Every object that is warmer than absolute zero gives off invisible **heat radiation** (infrared light). Thermal cameras do not see colour; they measure this radiation. Your task is to write a program that **fakes what a thermal camera would see**, using real physics, and then show it inside **Unity**, a game engine.

Why is this useful? Companies train and test thermal-camera software (drones, vehicles, security) in simulation because real thermal data is expensive and hard to collect. The simulation must be physically correct, not just "hot = red".

The pipeline in one line: temperature of each pixel + material property (emissivity) → Planck's law → add up over the 8–12 µm band → save as an image → load in Unity.

1.2 Key words explained simply

Term	Simple meaning	Example
Infrared (IR)	Light we cannot see, felt as heat	Warmth from a hot cup
Wavelength (λ)	The "size" of a light wave, measured in metres or micrometres (µm). 1 µm = 0.000001 m	8 µm = 8e-6 m (write it like this in Python)
LWIR	Long-Wave Infrared: 8–12 µm. **This is your band.**	Common thermal-camera range
MWIR	Mid-Wave Infrared: 3–5 µm. Crossed out in your document, so probably not needed	Confirm with your guide
Kelvin (K)	Temperature scale that starts at absolute zero. K = °C + 273.15	22 °C = 295 K; 60 °C = 333 K
Radiance	How much heat light a surface sends toward the camera. Unit: W/(m^2·sr)	Hotter surface = higher radiance
Emissivity (ε)	How good a surface is at radiating, from 0 to 1	Plastic ≈ 0.9, polished steel ≈ 0.15
Planck's law	The formula giving radiance for each wavelength at a given temperature	See Part 2
Integration	Adding up many small slices to get a total	Summing radiance from 8 to 12 µm
Texture	An image that Unity wraps onto a 3D object. Here each pixel stores a radiance number	A 256×256 image of the bottle
EXR / 16-bit PNG	Image formats that keep precise numbers. EXR stores exact decimals (32-bit float); 16-bit PNG stores whole numbers 0–65535	Use EXR for accuracy, PNG as backup
Unity (HDRP)	Game engine; HDRP is its high-quality rendering pipeline	Version 2022 or newer
Shader	A small GPU program that decides each pixel's colour	Written in HLSL (Month 5)
DN (Digital Number)	Raw pixel value a sensor outputs (e.g. 0–65535). My reading of your guide's note; confirm it	The 16-bit PNG values
PBR	Physically Based Rendering: Unity's realistic material system	Mentioned in your guide's note
Virtual environment	A private box of Python packages for one project	The folder .venv
Git	Saves versions of your project so you can go back	git commit

> **NOTE:** Your task PDF has handwritten notes: "DN = Digital Number", "Metal-Steel-Wood", "3D object" and "PBR". I interpreted them as above. Please confirm with your guide because handwriting can be misread.

Part 2. The physics in simple words

2.1 Planck's law

For a surface at temperature **T**, the radiance at wavelength **λ** is:

$$L(\lambda, T) = 2hc^2 / [\, \lambda^5 \cdot (e^{hc/(\lambda kT)} - 1) \,]$$

- h = 6.626e-34 J·s (Planck constant)
- c = 2.998e8 m/s (speed of light)
- k = 1.381e-23 J/K (Boltzmann constant)
- Plain meaning: **hotter → more radiance**. At 8–12 µm a 300 K object is bright, which is why room-temperature things show up on thermal cameras.

2.2 Emissivity

A real material radiates only a fraction ε of what a perfect "blackbody" would: **R = ε × L(λ, T)**. Dull plastic is close to perfect (0.9). Shiny metal is poor (0.1–0.2) because it behaves like a mirror.

2.3 Adding up over the band

A camera does not measure one wavelength. It collects the whole 8–12 µm band. So we compute L at many wavelengths (every 0.01 µm gives 401 values) and add them with the **trapezoid rule**:

R_integrated = ε × ∫ L(λ, T) dλ (from 8 to 12 µm)

2.4 Worked example

A flat plate at 350 K with ε = 0.95 gives about **74 W/(m^2·sr)** in the 8–12 µm band (I calculated this numerically). If your program prints a value wildly different from that for this input, you probably have a unit mistake: wavelength must be in **metres** and temperature in **Kelvin**.

2.5 Why steel is special (reflection)

A surface that radiates poorly also **reflects** its surroundings well. So what the camera sees is:

Apparent radiance = ε × L(T_object) + (1 − ε) × L(T_room)

For polished steel (ε = 0.15), 85 % of what you see is a mirror image of the room. This is why a hot polished bottle does **not** look very hot on a thermal camera. The reflection term is **my addition**; your task PDF only has the first term. The script lets you switch it on or off.

Part 3. The existing procedure (from your task document)

This part only restates what your internship document already asks for. My suggestions are kept separate in Part 8.

3.1 Input, process, output

Input	Process	Output
Material data (spectral emissivity), temperature distribution, sensor band (LWIR 8–12 µm)	1) Compute Planck-based spectral radiance for each wavelength. 2) Integrate over the band (or keep the per-λ stack). 3) Encode into a high-dynamic-range texture (OpenEXR or 16-bit PNG).	A 2-D radiance texture to import in Unity for a custom IR-camera shader, with radiometric error ≤ 5 % against reference calculations

3.2 Success criteria

- Validation error ≤ 5 % (integrated radiance vs. analytical reference).
- Generation time ≤ 30 s for a 1024×1024 texture on a standard workstation GPU.
- Full pipeline (data → texture → Unity import) documented and reproducible.

3.3 Six-month work plan

Month	Focus	Key sub-steps	Deliverables
1	Requirements and environment setup	Collect datasheets (emissivity). Define temperature map formats. Install Python stack (NumPy, SciPy, imageio, OpenEXR) and Unity 2022+ HDRP. Create Git repo.	Approved specification. Working dev environment with a simple test scene.

Month	Focus	Key sub-steps	Deliverables
2	Physical model implementation	Planck function in Python. Wavelength grid (0.01 µm). Emissivity handling (constant, lookup table or spatial map).	planck_lam.py with unit tests. Sample emissivity CSV.
3	Temperature map generation	Read temperatures from CSV, image or procedural noise. Uniform and varying fields. Visualise in Matplotlib.	temp_map_generator.py (256, 512, 1024 px). Validation plots.
4	Radiance computation and integration	Vectorised L(λ,T) per pixel. Trapezoid integration. Optional per-λ stack.	radiance_generator.py writing EXR (32-bit) and PNG (16-bit). Accuracy report (MAE ≤ 5 %).
5	Unity integration and shader hook-up	Import texture (Single Channel, Linear). Minimal C# script. Custom HLSL shader. Pixel-wise comparison.	Prefab IRRadianceTexture. Shader IRRadianceShader.hlsl.
6	Optimisation, validation, documentation	Profile (Numba or CuPy) for ≤ 30 s. Test 3+ scenarios. 20-page report, 5-minute video, clean repo.	Final report and video. Hand-over repository. Optional note on atmospheric attenuation.

3.4 Your steel bottle as the first test case

The bottle is a simple way to practise the whole pipeline with **three materials** (background, steel body, plastic cap), each with its own emissivity and temperature. It matches the guide's notes about metal/steel and a 3D object.

Part 4. Hands-on: install and prepare data in parallel

> **TIP: Which terminal?** Use **Windows PowerShell** (Start menu → type PowerShell) for Stages 1–4. From Stage 5 you can use the **VS Code terminal** instead. On Windows the VS Code terminal is PowerShell by default, so **every command in this guide is identical in both**. Run one line at a time and press Enter.

Stage 1. Start the downloads

TRACK A – INSTALL | Check what is already installed, then install the rest

```
winget --version
python --version
git --version
```

If **winget** is not recognised, install "App Installer" from the Microsoft Store. If python or git is not recognised, install them (accept any prompts):

```
winget install Python.Python.3.11
winget install Git.Git
winget install Microsoft.VisualStudioCode
```

When finished, **close PowerShell and open a new window**. This refreshes the PATH, which is where Windows looks for programs.

TRACK B – PREPARE BOTTLE DATA | Step B1: decide what your bottle is (while installs run)

Write these answers in Notepad:

1. Is the bottle **polished** stainless steel, **brushed** steel, or **painted / powder-coated**? This changes emissivity a lot.
2. What is inside: hot water, cold water, or empty?
3. What is the room temperature (for example 22 °C)?

4. Is the cap plastic?

Stage 2. Verify the installs

TRACK A – INSTALL | In a NEW PowerShell window

```
python --version
git --version
code --version
```

Expected: Python 3.11.x, a Git version, and a VS Code version number.

TRACK B – PREPARE BOTTLE DATA | Step B2: choose the three input numbers for each part of the bottle

Part	Emissivity (ε)	Temperature	Why
Background wall	0.95	295 K (22 °C)	Typical room wall, close to a perfect radiator
Steel body	0.15 (polished)	333 K (60 °C)	Hot water inside, single-wall bottle
Plastic cap	0.90	305 K (32 °C)	Warmed by the bottle, plastic radiates well

Typical emissivity values for steel (rough numbers; confirm with the datasheet your guide provides): polished stainless ≈ 0.1–0.15, brushed ≈ 0.2–0.4, oxidised ≈ 0.6–0.8, painted ≈ 0.9.

Convert to Kelvin first: 22 °C + 273.15 = 295.15 K, rounded to 295 K. 60 °C = 333.15 K, rounded to 333 K.

> **TIP:** Real insulated bottles stay near room temperature on the outside. For a second scenario, set the steel body to 297 K to represent a "cold-wall" insulated bottle.

Stage 3. Create the project, the environment, and install packages

TRACK A – INSTALL | Folder, Git, virtual environment

```
cd $HOME\Documents
mkdir ir-radiance
cd ir-radiance
git init
python -m venv .venv
.\.venv\Scripts\Activate.ps1
```

Your prompt should now start with **(.venv)**. That means the private package box is active.

> **WATCH OUT:** If PowerShell says "running scripts is disabled", run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`, answer Y, then run the Activate line again.

Now install the Python packages (this takes a few minutes):

```
python -m pip install --upgrade pip
pip install numpy scipy matplotlib imageio pytest numba openexr
pip freeze > requirements.txt
```

TRACK B – PREPARE BOTTLE DATA | Step B3: create the input file (open a SECOND PowerShell tab)

Open a second PowerShell window or tab and run:

```
cd $HOME\Documents\ir-radiance
mkdir data
mkdir output
@"
material_id,name,emissivity,temperature_K
0,background_wall,0.95,295
1,steel_body,0.15,333
```

```
2,cap_plastic,0.90,305
"@ | Set-Content data\materials.csv
```

This CSV is your **material data and temperature input**. Edit the numbers later to test other scenarios.

> **TIP: Optional, in parallel:** start downloading Unity Hub and a Unity 2022 LTS editor with the HDRP template from unity.com/download. It is large and you only need it from Month 5. Ask your guide which exact Unity version to use.

Stage 4. Verify the packages

TRACK A – INSTALL | Back in the (.venv) window

```
python -c "import numpy, scipy, matplotlib, imageio, numba; print('numpy', numpy.__version__, '| scipy', scipy.__version__,
python -c "import OpenEXR; print('OpenEXR OK')"
```

If the OpenEXR line fails, do not panic. The 16-bit PNG still works, and the script skips EXR safely. Tell your guide.

TRACK B – PREPARE BOTTLE DATA | Step B4: understand the bottle shape map

The program draws the bottle on a 256×256 grid. Each cell (pixel) gets a **material ID**:

ID	Meaning	Grid area in the script (rows : columns)
0	Background wall	Everything else
1	Steel body	rows 90 to 229, columns 96 to 159 (64 wide, 140 tall)
2	Plastic cap	rows 60 to 89, columns 116 to 139 (24 wide, 30 tall)

Row numbers go **down** the image from the top, column numbers go **across** from the left. To match your real bottle, measure its width and height and keep the same ratio (for example 7 cm × 20 cm means the body is about 0.35 times as wide as it is tall).

Stage 5. Open the project in VS Code

TRACK A – INSTALL | Start VS Code from the project folder

```
code .
```

1. Click the **Extensions** icon (left bar) and install **Python** (by Microsoft).
2. Press **Ctrl+Shift+P**, type **Python: Select Interpreter**, and choose the entry ending in **.venv\Scripts\python.exe**.
3. Open the terminal with **Ctrl+`** (backtick key). A new terminal auto-activates the venv. Check for **(.venv)**.
4. Verify again here: **python --version** and **pip list**.

If **code .** is not recognised, open VS Code from the Start menu, then File → Open Folder → Documents\ir-radiance.

TRACK B – PREPARE BOTTLE DATA | Step B5: choose your test scenarios

Scenario	Steel ε	Steel T	What you learn
1. Hot polished bottle (baseline)	0.15	333 K	Reflection dominates; steel looks dull
2. Hot brushed bottle	0.30	333 K	More emission, brighter
3. Insulated bottle (cold wall)	0.15	297 K	Almost invisible against the room
4. Painted bottle	0.90	333 K	Looks hot and bright

Run scenario 1 first. For the others, edit only the steel row in **data\materials.csv** and run again.

Part 5. The Python script and running it

5.1 Create the file

In VS Code: Explorer → New File → name it **bottle_radiance.py** (in the ir-radiance folder), then paste the code below. Quick check: the folder should now contain **data\materials.csv**, **output**, and **bottle_radiance.py**.

```
import csv, json
import numpy as np
import matplotlib
matplotlib.use("Agg")                 # save plots as files, no pop-up window
import matplotlib.pyplot as plt
import imageio.v3 as iio
from scipy.integrate import quad

h, c, kb = 6.626e-34, 2.998e8, 1.381e-23
LAM_MIN, LAM_MAX, DL = 8e-6, 12e-6, 0.01e-6
REFLECT = True                        # add reflected room radiation

def planck_lam(lam, T):
    """Spectral radiance in W / (sr * m^3)."""
    x = h * c / (lam * kb * T)
    return (2 * h * c**2) / lam**5 / np.expm1(x)

def band_radiance(T_map, eps=1.0):
    """Trapezoid integral over the band, memory-friendly (loops over wavelength)."""
    n = round((LAM_MAX - LAM_MIN) / DL) + 1
    lam = np.linspace(LAM_MIN, LAM_MAX, n)
    out = np.zeros_like(T_map, dtype=np.float64)
    prev = planck_lam(lam[0], T_map)
    for i in range(1, n):
        cur = planck_lam(lam[i], T_map)
        out += 0.5 * (prev + cur) * (lam[i] - lam[i - 1])
        prev = cur
    return eps * out

# 1. read inputs
mats = {}
with open("data/materials.csv") as f:
    for r in csv.DictReader(f):
        mats[int(r["material_id"])] = (float(r["emissivity"]), float(r["temperature_K"]))

# 2. build bottle shape (0 = background, 1 = body, 2 = cap)
H = W = 256
mat = np.zeros((H, W), dtype=int)
mat[90:230, 96:160] = 1
mat[60:90, 116:140] = 2

# 3. temperature and emissivity maps
T_map = np.zeros((H, W)); eps_map = np.zeros((H, W))
for m, (e, t) in mats.items():
    T_map[mat == m] = t
    eps_map[mat == m] = e
T_amb = mats[0][1]

# 4-5. radiance
R = band_radiance(T_map, eps_map)
if REFLECT:
    L_amb = band_radiance(np.array([T_amb]))[0]
    R = R + (1 - eps_map) * L_amb

# 6. save
scale = float(R.max())
iio.imwrite("output/bottle_radiance.png", (R / scale * 65535).astype(np.uint16))
json.dump({"scale_max_W_per_m2_sr": scale}, open("output/scale.json", "w"))
try:
    import OpenEXR
    header = {"compression": OpenEXR.ZIP_COMPRESSION, "type": OpenEXR.scanlineimage}
    with OpenEXR.File(header, {"Y": R.astype(np.float32)}) as f:
        f.write("output/bottle_radiance.exr")
```

```
    print("EXR saved")
except Exception as e:
    print("EXR skipped:", e)

# 7. plots
fig, ax = plt.subplots(1, 3, figsize=(13, 4))
titles = ["Temperature (K)", "Emissivity", "Radiance (W/m2/sr)"]
for a, d, t in zip(ax, [T_map, eps_map, R], titles):
    im = a.imshow(d, cmap="inferno"); a.set_title(t); plt.colorbar(im, ax=a)
plt.savefig("output/overview.png", dpi=120)

# validation: compare steel-body pixel to scipy.quad
e, t = mats[1]
ref = e * quad(lambda l: planck_lam(l, t), LAM_MIN, LAM_MAX)[0]
if REFLECT:
    ref += (1 - e) * quad(lambda l: planck_lam(l, T_amb), LAM_MIN, LAM_MAX)[0]
mine = R[150, 128]
print(f"mine={mine:.3f}  reference={ref:.3f}  error={abs(mine-ref)/ref*100:.4f} %")
print(f"max radiance = {scale:.2f} W/m2/sr")
```

WATCH OUT: Copying code from a PDF can break quotes or line breaks. The same script is supplied as a separate file, **bottle_radiance.py**, next to this PDF. Use that file if you get odd syntax errors.

5.2 What the script does, step by step

1. **Reads** emissivity and temperature per material from the CSV.
2. **Draws** the bottle as a grid of material IDs.
3. **Builds** a temperature map and an emissivity map (one value per pixel).
4. **Computes** Planck radiance at 401 wavelengths and adds them with the trapezoid rule. It loops over wavelengths so memory stays small.
5. **Adds reflection** of the room if REFLECT is True.
6. **Saves** a 16-bit PNG, a scale.json (needed to convert PNG values back to real units), and an EXR if available.
7. **Plots** temperature, emissivity and radiance side by side.
8. **Validates** one steel pixel against SciPy's independent integration (quad).

5.3 Run it

In PowerShell or the VS Code terminal, inside the ir-radiance folder with (.venv) active:

```
python bottle_radiance.py
```

Expected output (I ran this exact script with these inputs; the EXR line depends on whether OpenEXR installed):

```
EXR saved            (or: EXR skipped: ...)
mine=39.579  reference=39.579  error=0.0000 %
max radiance = 41.15 W/m2/sr
```

Then open **output\overview.png**. It should look like the figure below: the bottle on a uniform background, with the steel body having **low emissivity** but, because of reflection, still not dark.

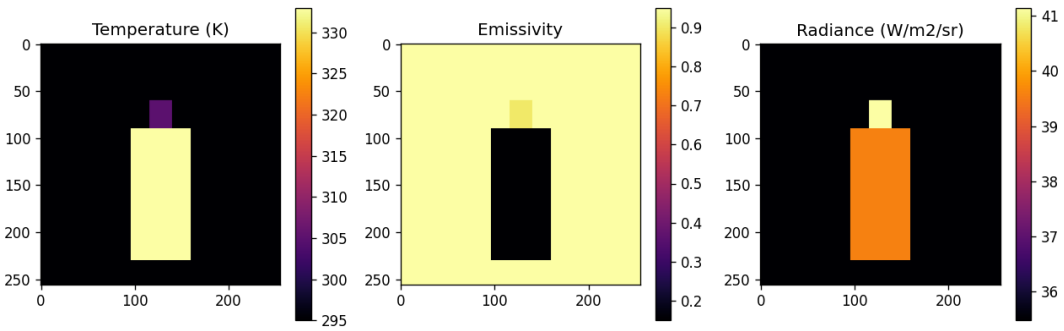


Figure: left = temperature map, middle = emissivity map, right = radiance map (produced by the script).

5.4 Radiance values you should get

Region	ε	T (K)	REFLECT = False	REFLECT = True
Background wall	0.95	295	33.7	35.5
Steel body	0.15	333	9.4	39.6
Plastic cap	0.90	305	37.6	41.1

Units: W/(m^2·sr). Look at the steel body: **without reflection it is the darkest part (9.4)**, even though it is the hottest at 333 K. With reflection of the room it jumps to 39.6. This is the main lesson of the bottle experiment. Change `REFLECT = True` to `False` in the script and run again to see it.

5.5 Save your work with Git

```
git add .
git commit -m "First steel bottle LWIR radiance script"
```

If Git asks who you are, run these once (use your own name and email):

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

5.6 Preview of later steps (Month 5): using the texture in Unity

1. Import the EXR (preferred, full dynamic range) or the PNG into Unity. Set the import type to **Single Channel** and **Linear** (not sRGB).
2. Attach a small C# script to a GameObject to read the pixel values.
3. Pass the data to a custom HLSL shader that applies the sensor response.

```
public Texture2D radianceTex;      // assign in inspector

void Start() {
    var data = radianceTex.GetPixelData<float>(0);
    // Convert to a ComputeBuffer or pass as a texture to the shader
}
```

Part 6. Testing, accuracy and performance

6.1 Test 1: automatic unit test for the Planck function

Create a file **test_planck.py** in the project folder:

```
import numpy as np
from bottle_radiance import planck_lam

def test_total_radiance_matches_stefan_boltzmann():
    lam = np.linspace(1e-6, 1e-3, 400000)
    trap = getattr(np, "trapezoid", None) or np.trapz
    total = trap(planck_lam(lam, 300.0), lam)
    expected = 5.670e-8 * 300**4 / np.pi
    assert abs(total - expected) / expected < 0.01

pytest
```

This test adds up Planck radiance over *all* wavelengths and compares it with a famous independent formula (Stefan–Boltzmann: $\sigma T^4/\pi \approx 146$ W/(m^2·sr) at 300 K). Passing means your Planck function is physically correct. Note: importing the module re-runs the whole script, which is fine for now; later you can tidy the code into functions.

6.2 Test 2: accuracy against an independent reference (target ≤ 5 %)

- The script already prints **error = ... %** comparing your trapezoid result with SciPy's quad. It should be far below 5 % (close to 0 %).
- Compute **error = |yours − reference| / reference × 100**.
- Test extremes: 300 K and 500 K, and ε = 0.1 and 0.95.
- Test the grid step: try DL = 0.05e-6. If the error is still tiny, you can save time.
- Reload the saved PNG and multiply back by scale.json to check the 16-bit rounding error is tiny (well under 0.01 %).
- In Unity (Month 5), sample a few pixels and compare with the Python numbers (pixel-wise).

6.3 Test 3: speed (target ≤ 30 s for 1024×1024)

Set `H = W = 1024` and multiply the bottle coordinates by 4, then time the run:

```
python -c "import time, subprocess; t=time.perf_counter(); subprocess.run(['python','bottle_radiance.py']); print(round(time
```

If it is too slow: use float32, use fewer wavelengths, use the lookup-table idea in Part 8, or accelerate with Numba or CuPy (your Month 6 task).

> **WATCH OUT: Memory warning.** A naive version that builds all wavelengths at once for 1024×1024 pixels needs about 1024×1024×401×8 bytes = **3.4 GB**. The script above avoids this by looping over wavelengths.

6.4 Test 4: three end-to-end scenarios

Run the scenarios from Step B5, save the three **overview.png** images under different names, and compare them. Your task requires at least three temperature/emissivity scenarios in Month 6.

Part 7. Troubleshooting

Problem	What it means	Fix
'python' is not recognized	Python not installed or PATH not refreshed	Reinstall Python (tick "Add to PATH") or open a NEW terminal
Activate.ps1 cannot be loaded	PowerShell blocks scripts by default	Set-ExecutionPolicy -Scope CurrentUser RemoteSigned, then activate again
No (.venv) in the prompt	Environment not active	Run .\.venv\Scripts\Activate.ps1 from the project folder
ModuleNotFoundError: numpy	Packages installed in a different Python	Activate .venv, or select the .venv interpreter in VS Code; run pip install again
pip install fails, SSL or timeout	Network, VPN or proxy problem	Check connection; try another network; run the command again
OpenEXR will not install	Package not available for your Python	Skip it for now (PNG works). Ask your guide for an EXR alternative
FileNotFoundError: data/materials.csv	You are in the wrong folder	Run pwd, then cd $HOME\Documents\ir-radiance
code is not recognized	VS Code PATH not set	Open VS Code from Start menu, use File → Open Folder
np.trapz not found or deprecated	Newer NumPy renamed it	Use np.trapezoid (the test file handles both)
Radiance values huge or tiny	Unit mistake	Wavelength in metres (8e-6), temperature in Kelvin
Overflow warning in exp	Very short wavelength or very low T	Stay in 8–12 µm and realistic temperatures (200–600 K)

Problem	What it means	Fix
Plot window does not appear	Script saves plots to files on purpose	Open output\overview.png
git commit asks for name and email	Git identity not set	Run the two git config commands in Part 5.5

TIP: A good habit: when something fails, copy the **last 5 lines** of the error message and read the very last line first. It usually names the exact problem. Send those lines to your guide or to me.

Part 8. Suggested improvements (NOT in the task document)

WATCH OUT: Everything in this part is my suggestion. It is **not** required by your internship document. Discuss it with your guide before spending time on it.

8.1 Improvement ideas

1. **Temperature lookup table (LUT).** Radiance depends only on T and ε, so compute R(T) once for 250–600 K in 0.1 K steps, then convert every pixel by interpolation. Much faster than integrating per pixel.
2. **Memory-safe computation.** Loop over wavelengths or use chunks and float32 (already done in the script).
3. **Material ID map with an emissivity table** (steel, wood, concrete, plastic). One texture can then hold many materials.
4. **Reflected radiation:** apparent = ε·L(T) + (1 − ε)·L(T_room). Essential for low-emissivity metals (already in the script, switchable).
5. **Save the scale factor** with the PNG (scale.json) so absolute units are never lost.
6. **Sensor spectral response and noise** applied in the shader (your PDF lists this as an optional extension).
7. **Atmospheric attenuation** as future work (also mentioned in your plan).

8.2 Best idea to discuss with your guide

IDEA: Store temperature (and material ID) in the texture, and convert to radiance in the Unity shader using a 1-D LUT texture. Why it is strong: it is fast (easily meets the 30 s target), it allows live temperature changes in Unity, it fits the optional "spectral response LUT" in your PDF, and the 5 % accuracy check stays the same. Keep the baked EXR as the reference output so the original deliverables are still met.

How to propose it: "I finished a working baseline that bakes radiance for a steel bottle with 0 % integration error against SciPy. I would also like to try a temperature plus LUT approach so the shader can change temperatures in real time. Is that in scope?"

Part 9. Summary and questions for your guide

9.1 One-page summary

- **Goal:** a Python tool that turns temperature maps and material emissivity into LWIR (8–12 µm) radiance images, saved as EXR/PNG and shown in Unity through a custom shader.
- **Physics:** Planck's law gives radiance per wavelength; multiply by emissivity; add up over 8–12 µm; for shiny steel add reflected room radiation.
- **Targets:** error ≤ 5 %, generation ≤ 30 s for 1024×1024, fully documented pipeline.
- **Setup:** Python 3.11, Git, VS Code, a virtual environment, and numpy, scipy, matplotlib, imageio, pytest, numba, openexr. Unity comes later.
- **Steel bottle data:** a CSV with three materials (ε and T) plus a grid map of material IDs.
- **Lesson from the bottle:** a hot polished steel bottle can look *darker* than a cooler plastic cap because of its low emissivity.

9.2 Questions to ask your guide

1. MWIR is crossed out in the document. Is the project LWIR only?
2. What does "DN" mean here, and should the PNG output be a calibrated DN?
3. Which materials (steel, wood, others) should I start with, and is there an emissivity data source?
4. Is the temperature data given, or should I generate it? Is the 3D object already available?
5. What is the "reference" for the 5 % error: analytical integration, or another tool?
6. Should I store radiance or temperature in the texture? (Mention the LUT idea.)
7. Is the sensor spectral response curve available, or should I assume a flat response?
8. Which GPU and which Unity version will we use?
9. Should reflected ambient radiation be included in the model?
10. Does the PBR note mean I should plug this into Unity's PBR material workflow?

Appendix A. Quick command reference

"PS" means Windows PowerShell or the VS Code terminal (they behave the same). Always run commands from the folder shown.

Purpose	Command	Run in
Check tools	winget --version / python --version / git --version / code --version	PS, anywhere
Install tools	winget install Python.Python.3.11 / Git.Git / Microsoft.VisualStudioCode	PS, anywhere
Go to Documents	cd $HOME\Documents	PS
Make project folder	mkdir ir-radiance ; cd ir-radiance	PS
Start Git	git init	PS, project folder
Create environment	python -m venv .venv	PS, project folder
Activate environment	.\.venv\Scripts\Activate.ps1	PS, project folder
Allow scripts	Set-ExecutionPolicy -Scope CurrentUser RemoteSigned	PS, anywhere
Upgrade pip	python -m pip install --upgrade pip	PS, (.venv) active
Install packages	pip install numpy scipy matplotlib imageio pytest numba openexr	PS, (.venv) active
Save package list	pip freeze > requirements.txt	PS, (.venv) active
Open VS Code	code .	PS, project folder

Purpose	Command	Run in
Run the project	python bottle_radiance.py	PS, (.venv) active, project folder
Run tests	pytest	PS, (.venv) active, project folder
Save a version	git add . ; git commit -m "message"	PS, project folder
Leave the environment	deactivate	PS

Daily routine: open VS Code → open the ir-radiance folder → open a new terminal → check for (.venv) → run python bottle_radiance.py → look at output\overview.png → git commit.